Esercitazione 29 Aprile

Template di classi

Un po' di teoria [*Prova pratica del 15 gennaio 2020, Testo 1, Esercizio 1*]

Identificare e correggere gli errori nella seguente dichiarazione di template di classe C++:

```
1       template <typename T>
2       Vettore {
3       public:
4               Vettore();
5               Vettore(int dim, T* pval);
6               Vettore(const Vettore<T>& v);
7               ~Vettore() const;
8               int getDim();
9               const T* getValues() const;
10              T getSingleValue(int index) const;
11              void setValues(T* pvals);
12              void setSingleValue(int index, T val);
13              Vettore<T> operator+(const Vettore<T>& v); // operatore binario +
14              Vettore<T>& operator=(const v);
15      public:
16              int _dim;
17              T* _val;
18 };
```

Un po' di teoria [*Prova pratica del 15 gennaio 2020, Testo 1, Esercizio 1*]

Identificare e correggere gli errori nella seguente dichiarazione di template di classe C++:

```
1   template <typename T>
2   Vettore {
3   public:
4           Vettore();
5           Vettore(int dim, T* pval);
6           Vettore(const Vettore<T>& v);
7           ~Vettore() const;
8           int getDim();
9           const T* getValues() const;
10          T getSingleValue(int index) const;
11          void setValues(T* pvals);
12          void setSingleValue(int index, T val);
13          Vettore<T> operator+(const Vettore<T>& v); // operatore binario +
14          Vettore<T>& operator=(const v);
15  public:
16          int _dim;
17          T* _val;
18 };
```

3

Un po' di teoria [*Prova pratica del 15 gennaio 2020, Testo 1, Esercizio 1*]

Identificare e correggere gli errori nella seguente dichiarazione di template di classe C++:

```
1   template <typename T>
2   class Vettore {
3   public:
4           Vettore();
5           Vettore(int dim, T* pval);
6           Vettore(const Vettore<T>& v);
7           ~Vettore() ~~const~~;
8           int getDim() const;
9           const T* getValues() const;
10          T getSingleValue(int index) const;
11          void setValues(T* pvals);
12          void setSingleValue(int index, T val);
13          Vettore<T> operator+(const Vettore<T>& v); // operatore binario +
14          Vettore<T>& operator=(const Vettore<T>& v);
15  private ~~public~~:
16          int _dim;
17          T* _val;
18 };
```

4

Un po' di teoria [*Prova pratica del 24 luglio 2017, Testo 2, Esercizio 1*]

Completare il codice dell'interfaccia del seguente template di classe C++:

```
1. // classe per rappresentare parabole secondo l'equazione y = ax² + bx + c.
2. ____________________ <typename T>
3. class Parabola {
4. ____________________ :
5.     Parabola();
6.     Parabola(T a, T b, T c);
7.     ____________________ (const Parabola<T>& p);
8.     ~Parabola();
9.     T getA() const;
10.    T getB() const;
11.    T getC() const;
12.    void setData(T a, T b, T c);
13.    ____________________ operator=(const Parabola<T>& p);
14. private:
15.    T _a, _b, _c ;
16.};
```

5

Un po' di teoria [*Prova pratica del 24 luglio 2017, Testo 2, Esercizio 1*]

Completare il codice dell'interfaccia del seguente template di classe C++:

```
1. // classe per rappresentare parabole secondo l'equazione y = ax² + bx + c.
2. template <typename T>
3. class Parabola {
4. public :
5.     Parabola();
6.     Parabola(T a, T b, T c);
7.     Parabola (const Parabola<T>& p);
8.     ~Parabola();
9.     T getA() const;
10.    T getB() const;
11.    T getC() const;
12.    void setData(T a, T b, T c);
13.    Parabola<T>& operator=(const Parabola<T>& p);
14. private:
15.    T _a, _b, _c ;
16. };
```

6

[Esercizio C082] Tripla

- Implementa una classe template chiamata `Tripla` che rappresenti una tripla di elementi dello stesso tipo. La classe deve fornire le seguenti funzionalità:
  - Un costruttore che permetta di inizializzare la tripla con tre valori dello stesso tipo.
  - Un metodo `trovaMassimo()` che restituisca il valore massimo tra i tre elementi della tripla.
  - Un metodo `trovaMinimo()` che restituisca il valore minimo tra i tre elementi della tripla.
  - Un metodo `ordina()` che ordini i tre elementi della tripla in ordine crescente.
  - Un metodo stampaTripla() che stampa i valori di 3 elementi.
- Infine, scrivi un programma principale per testare la classe `Tripla` con almeno due tipi semplici di dati diversi.

7

[Esercizio C083] Estensione di Tripla

- Crea una classe Persona (con degli attributi: nome, cognome, codice fiscale, età, gender)
  - sovraccarica gli operatori necessari per poter svolgere le operazioni su una tripla di persone (come trovare massimo, minimo…), utilizza l'attributo *età* per fare il confronto fra due persone
- Nel main:
  - utilizza il template creato in C082
  - crea una tripla di persone
  - ordina una tripla di persone

8

[Esercizio C086] - Image processing

- Implementare una classe template Matrice per rappresentare una matrice di elementi generici.
  - aggiungere alla classe Matrice un metodo getElemento() che restituisce un elemento specifico della matrice e setElemento() per inserire un *elemento*,
  - sovraccaricare il operatore di moltiplicazione per fare una moltiplicazione scalare.
  - Definire la classe Pixel che rappresenta un colore nello standard RGB (tre attributi interi).
- Nel main:
  - creare una matrice di interi 4x4 usando il template e riempirla con valori casuali;
  - moltiplicare la matrice di interi per uno scalare e stampare la matrice risultante;
  - creare una matrice di pixel 5x5 usando il template e riempirla con colori casuali;
  - moltiplicare la matrice di pixel per uno scalare e stampare la matrice risultante;

9

[Esercizio C081] Esercizio d'esame (07 giugno 2017, Testo 1, Esercizio 5)

Allo scopo di sviluppare un software per la geometria analitica si realizzi in linguaggio C++ il template di classe *Retta*, parametrizzato sul tipo T ed avente per attributi il coefficiente angolare *_m* e l'intercetta *_q* (entrambi di tipo T) di una retta sul piano cartesiano, secondo l'equazione $y = mx + q$. Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi il valore di *_m* e di *_q* a zero, utilizzando la lista di inizializzazione.
- Il costruttore con parametri che riceva come parametri il valore del coefficiente angolare *m* e dell'intercetta *q* (entrambi di tipo T) ed inizializzi i corrispondenti attributi della classe usando la lista di inizializzazione.
- Il costruttore di copia.
- Il distruttore
- I selettori (un metodo per ciascun attributo)
- I modificatori (un metodo per ciascun attributo)
- Il metodo *inserisciDati* per leggere da tastiera i valori del coefficiente angolare _m e dell'intercetta _q. Il metodo non restituisce alcun valore di ritorno.
- Il metodo *stampaDati* che stampi a video l'equazione della retta secondo il seguente formato: *r*: $y = mx + q$; dove a *m* viene sostituito il valore corrente di *_m* e a *q* il valore corrente di *_q*. Il metodo non restituisce alcun valore di ritorno.

10

[Esercizio C081] Esercizio d'esame (07 giugno 2017, Testo 1, Esercizio 5)

- Il metodo *calcolaDistanza* che riceva come parametri le coordinate (*x*P, *y*P) di un punto P (due valori di tipo T), calcoli e restituisca come valore di ritorno la distanza *d* di P dalla retta (un numero reale), calcolata come:

$$d = \frac{\left|y_P - (mx_p + q)\right|}{\sqrt{1 + m^2}}$$

Si sviluppi, infine, la funzione *main* che operi nel modo seguente:

- Dopo aver chiesto all'utente di inserire da tastiera un numero intero *n* e avere verificato che *n* sia positivo, allochi dinamicamente un array R di *n* oggetti di classe *Retta,* assegnando il tipo *double* al parametro T. Nel caso in cui *n* sia negativo, *main* terminerà stampando a video un messaggio di errore.
- Scandisca l'array R e assegni i valori di *_m* e *_q* a ciascun elemento di R, chiamando il metodo *inserisciDati*.
- Chieda all'utente di inserire le coordinate (x_Q, y_Q) di un punto Q (due numeri reali), scandisca l'array R, invochi su ciascun elemento dell'array R la funzione *calcolaDistanza* e ne stampi a video il valore di ritorno (ovvero la distanza del punto Q da ciascuna delle rette contenute nell'array R).
- Deallochi l'array R e termini.

11

[Esercizio C085] Reimplementare gli array 1D

Si progetti e si implementi in linguaggio C++ il template di classe *Array* per realizzare un array contenente *n* oggetti generici di tipo T. Si supponga che il tipo T sia dotato degli operatori di assegnamento, di inserimento e di estrazione. Il template di classe avrà i seguenti attributi: un puntatore *_pdata* a un array di oggetti di tipo T e la dimensione *_dim* dell'array, cioè il numero di oggetti di cui l'array si compone (un numero intero). Il template di classe dovrà inoltre fornire i seguenti metodi:

- Il costruttore di default che inizializzi *_pdata* a NULL e *_dim* a zero.
- Un costruttore con parametri che riceva in ingresso (ovvero come parametro) un numero intero *n* e operi come segue: verifichi che *n* sia un numero positivo e, in tal caso, allochi dinamicamente un array di *n* oggetti di tipo T, ne assegni il puntatore a *_pdata* e assegni a *_dim* il valore *n*. Nel caso in cui *n* sia negativo o nullo, il costruttore inizializzerà *_pdata* a NULL e *_dim* a zero.
- Il costruttore di copia
- Il distruttore.
- Il selettore *get_size* che restituisca la dimensione *_dim* dell'array (un numero intero).
- Il selettore *get_value* che riceva come parametro un numero intero *index* e, dopo aver verificato che *index* sia compreso tra 0 e *_dim* – 1, restituisca come valore di ritorno l'elemento dell'array di indice *index* (di tipo T). Nel caso in cui il valore di *index* non sia valido o il puntatore *_pdata* sia nullo, il selettore restituisce NULL.

12

[Esercizio C085] Reimplementare gli array

- Il selettore *get_array* che restituisca come valore di ritorno il puntatore al primo elemento dell'array (un puntatore a costante di tipo T).
- Il modificatore *set_size* che riceva come parametro un numero intero *n*, restituisca come valore di ritorno un numero intero e modifichi l'array in modo tale che esso contenga *n* elementi. Il modificatore potrà utilizzare la tecnica di riallocazione già incontrata nelle precedenti esercitazioni. In particolare, il metodo allocherà un nuovo array di dimensione *n*, vi copierà gli elementi contenuti nell'array puntato da *_pdata* (nel caso in cui la nuova dimensione *n* sia più piccola della precedente, gli elementi eccedenti andranno perduti), deallocherà l'array puntato da *_pdata* e assegnerà a *_pdata* il puntatore al nuovo array. Al termine della riallocazione, il modificatore assegnerà a *_dim* la nuova dimensione *n*. Nel caso in cui il valore di *n* non sia valido o la riallocazione non vada a buon fine, il metodo non modifica l'array e restituisce 0, altrimenti restituisce 1.
- Il modificatore *set_value* che riceva come parametro un numero intero *index* e un oggetto *data* di tipo T e restituisca come valore di ritorno un numero intero. Dopo aver verificato che *index* sia compreso tra 0 e *_dim* – 1, il modificatore assegna l'oggetto T all'elemento dell'array di indice *index*. Nel caso in cui il valore di *index* non sia valido o il puntatore *_pdata* sia nullo, il modificatore restituisce 0, altrimenti restituisce 1.

13

[Esercizio C085] Reimplementare gli array

- L'operatore di assegnamento.
- Il metodo *print* che scandisca l'array puntato da *_pdata* e ne stampi a video gli elementi.
- Il metodo *read* che acquisisca da tastiera i valori degli elementi contenuti nell'array puntato da *_pdata*.

Si scriva quindi un programma C++ per verificare il corretto funzionamento del template di classe.

- Il programma dichiarerà un oggetto *a* di tipo *array*, istanziandone il tipo generico con il tipo *int* e utilizzando il primo costruttore con parametri passando la dimensione *n* = 10,
- chiederà all'utente di inserire da tastiera gli elementi dell'array *a* utilizzando il metodo *read*,
- stamperà quindi l'intero array utilizzando il metodo *print*,
- infine scandirà l'array calcolando e stampando a video la somma dei valori in esso contenuti.

Provate infine ad aggiungere al template di classe *array* l'operatore di selezione con indice []. Il template di classe *array* così realizzato permette di svolgere le stesse operazioni normalmente supportate dagli array predefiniti?

14